

they are completed and run in a single thread.

2. **Asynchronous API:** Call backs are specified using threads, to notify clients when the above operations are completed using the concerned acknowledgements.

Building a Paho C library

First, type the following commands:

```
git clone https://github.com/eclipse/paho.mqtt.c
cd paho.mqtt.c
cmake . #may go for a clean directory
make
make install
```

This builds and installs four types of libraries in `/usr/local/lib`:

- `libpaho-mqtt3c.*` for synchronous clients over TCP
- `libpaho-mqtt3cs.*` for synchronous clients over SSL
- `libpaho-mqtt3a.*` for asynchronous clients over TCP
- `libpaho-mqtt3as.*` for asynchronous clients over SSL

You can also find various reference examples and tools installed in `/usr/local/bin`. `paho_c_pub`, `paho_c_sub` are generic synchronous clients, which take the topic name as argument; they take options like `--host`, `--port`, `--qos`, `--keepalive`, `--clientid`, `--username`, `--password` for both the clients and `--retained` for the publisher.

Mosquitto clients

You can also obtain clients by building the Mosquitto source as mentioned above. These are available as `mosquitto_pub`, `mosquitto_sub` and installed to `/usr/local/bin`, which are copied from the `client` sub-directory during installation. Here is how these tools are used:

```
mosquitto_pub -h hostname -p portno -t topicname -m payload
mosquitto_sub -h hostname -p portno -t topicfilter
```

Other important options of these tools are listed below.

```
-i: client ID
-u: username
-p: password
-k: keep alive duration
-q: QoS level
-r: retain message
-l: payload from stdin(single line)
-s: payload from stdin(multi line)
--will-topic, --will-message, --will-retain: LWT support
-c: disabling clean session
```

By default, Mosquitto clients use v3.1. Specify `-V` `mqttv3.1.1` to enforce v3.1.1.

A few more client implementations are:

- Paho client libraries in many other languages
- Paho UI tool developed using Java Swing
- `mqtt.js` in Nodejs through npm also provides a simple command line tool

- NodeRED nodes for MQTT
- `mqtt-spy` in Java
- MQTTLens, a Chrome plugin

Android apps

There are many apps for talking to MQTT brokers. Among these, a few come with the source code. The sample app part of Eclipse Paho Android service is a good one to start I feel, and can be customised for your needs. For this, check out the code from <http://www.eclipse.org/paho/clients/android/sample/> and build using Android Studio or the Gradle build system, and deploy to the device.

Open the installed app and click on the '+' symbol to provide broker details and hit **Connect**. Once connected, use the UI features on the **publish**, **subscribe** tabs and play around. Delivered messages based on subscriptions are notified using toast messages and listed in the history tab. You may use the public broker available at test.mosquitto.org for quick testing.

MQTT broker on the cloud

Many cloud platforms support connectivity through MQTT with certain topic patterns and payloads in plain text or JSON format. A few such platforms are listed below. Apart from hosting brokers, they offer features like dashboards with graph plotting, Web integration and interoperability with other protocols, etc.

- IBM Bluemix IOT App
- Opensorsors.io
- ThingMQ
- Carriots
- CloudMQTT
- AWS IOT

...and many more. Please explore these platforms and their connectivity options.

MQTT over Websockets

MQTT can run over Websockets also, which enables browser based clients. Mosquitto, Surgemq, etc, support Websockets through a custom port along with the default TCP port. Paho provides JavaScript clients, which can be embedded in a Web page and use the WebSocket transport layer. Security over Websockets is still under evaluation with various brokers.

Using Wireshark for a better understanding of the protocol

Wireshark comes with MQTT support from version 1.12 onwards. Start Wireshark with suitable privileges, type `mqtt` in the filter box and hit **Enter**. Let's observe the packet flow and anatomy of packets using any broker and Mosquitto or Paho clients with a given syntax.

1. For each packet, observe the message type (packet type) in the fixed header and Msg Len (remaining length).
2. For each connected client, you can find the packet flow of **CONNECT**, **CONNACK** and **SUBSCRIBE**, **SUBACK** for each subscription.